Large language models: scale, steer, use

Summer School on AI for Economics and Finance · Day 3, Lecture 2

Anna Smirnova

Figures: Zhang et al., *Dive into Deep Learning* (d2l.ai, CC BY-SA); Raschka, *Build a Large Language Model (From Scratch)* (Manning, 2024); Brown et al. (2020). Structure follows Amidi & Amidi, *Super Study Guide*, ch. 4–5.

The plan

	part	question
0	Before you use one	encoder vs decoder as instruments; the embedding trap
1	Generating text	how do you get words out of $p(\text{next} \mid \text{context})$?
2	Pretraining at scale	what is it trained on, and what does size buy?
3	Scaling laws	how should a fixed budget be spent, and what does a "law" mean here?
4	From completer to assistant	SFT, preferences, RLHF, DPO
5	Adapting on a budget	prompting, LoRA, retrieval, when to use which
6	Evaluation and honest use	benchmarks, calibration, LLMs as annotators

> One model of the LLM as an object: a very large decoder transformer, pretrained on next-token prediction, then **tilted** toward behaviour people prefer. Everything else is engineering around that.

0. Before you use one

Three things about the object that decide how you should treat its output

Why economists reach for the encoder first: type discipline

An encoder is a **measurement instrument**:

$$\text{text} \longrightarrow \underbrace{\mathbf{h} \in \mathbb{R}^{768}}_{\text{fixed-length vector}} \longrightarrow \text{logit}(\hat{y})$$

That is the supervised-learning template you already hold — $X \to f \to \hat{y}$: a label set *you* defined, a loss you understand, a number you can put in a regression.

A decoder returns the **same type it was given**: a string. Nothing has been reduced. You still owe a parser, and the parser is where your assumptions hide.

The practical reasons stack on top:

- 110M parameters; fine-tunes in minutes, runs anywhere
- deterministic; weights you own; no API drift
- a 2019 recipe with a script (GLUE-era fine-tuning)
- outputs at least *shaped* like probabilities

BERT was the first model that made pretrained NLP **look like econometrics**. That is why it is the default, and mostly a good one.

A decoder as a classifier: the label is a word

"Decoder" is Vaswani's name from translation — the half that *emits target tokens* — not "decodes for humans".

Hold the decoder as a **conditional distribution over continuations**. Then classifying with it is not "ask and parse the answer"; it is a comparison:

$$p(\text{"positive"} \mid x) \quad \text{vs} \quad p(\text{"negative"} \mid x)$$

read straight off the next-token softmax. This is the *verbalizer* (Schick & Schütze 2020): the label set, expressed in words.

Seen this way, a decoder classifier **is an encoder whose labels are words** — and that explains zero-shot.

- BERT's classification head starts from **random weights**: it knows nothing about "positive" until you show it labels.
- The decoder's "head" is the vocabulary: the label words **carry pretraining knowledge** — what "positive", "hawkish", "material weakness" mean and what text they follow.

So the decoder trades your parser for its vocabulary. Prompt wording becomes part of the estimator; choose the label words, log them, test paraphrases.

The rookie mistake: raw embeddings all look alike

You take a pretrained model, mean-pool the last layer over your corpus, compute cosine similarities — and every pair scores **0.90–0.98**. Nearest neighbours are noise; clusters follow document length and boilerplate; TF-IDF beats you.

Anisotropy (Ethayarajh 2019; Gao et al. 2019): the softmax objective only cares about *differences* between logits, so its gradient pushes every hidden state in a shared direction. All vectors end up in a narrow cone far from the origin. Worse in top layers, worse on domain text (rare tokens, repeated boilerplate).

In my own testing, **PatentBERT** does this on patent abstracts: raw pooled vectors, random pairs at ~0.95.

The 10-second diagnostic: cosine between two *random* sentences from your corpus. Near 0, fine. Near 0.9, nothing downstream means anything yet.

Fixes, cheapest first

1. don't use the last layer; average the last few or probe to choose
2. **all-but-the-top** (Mu & Viswanath 2018): subtract the mean, drop the top 1–3 principal components
3. **whitening** (Su et al. 2021): covariance → identity
4. **SimCSE** (Gao et al. 2021): contrastive fine-tuning with dropout as the "positive"; no labels; directly restores uniformity
5. use a model that was **contrastively trained** (any sentence-transformer). Raw checkpoints are for generation and probing, never for cosine.

What people do with these objects: four applications

1. Vector search. Embed every document once with a contrastively trained model; a query is a dot product against the index. Runs at millions of documents per second on a laptop; the backbone of retrieval-augmented generation (RAG), which we only name today.

2. Measurement at scale. An LLM reads text into a *fixed schema* — sentiment, topic, exposure, "what is this patent for" — and the labels become a variable (part 6, with my own example). The encoder-vs-decoder choice from two slides ago is exactly the choice of instrument.

3. Embeddings as regressors — and their residuals. Didisheim, Kelly, Pourmohammadi & Tian (2025): embed each news article with an LLM; **residualize the embedding on the Jensen–Kelly–Pedersen firm characteristics**; the part that is *not* predictable from known factors — the "news shock" — more than doubles the return predictability of raw news and persists up to 18 months. The same move as this morning's rookie-mistake fix, with an economic model instead of a PCA doing the projecting.

4. Transformers on sequences that are not text. Discretize returns, order-book events or regimes into tokens and the whole machinery applies; what changes is the data-generating process — efficiency flattens the conditional mean, not the variance (Trojani, 15:30).

1. Generating text

Decoding: from a distribution to a sentence

Decoding: the model gives a distribution, you choose a rule

At each step the model outputs $p(x_t \mid x_{<t})$ over V tokens. Generation = pick a token, append, repeat until an end-of-text token.

Greedy: take the argmax each step. Deterministic, repetitive, and *not* the most likely sentence (a locally best token can lead to a bad continuation).

Beam search: keep the k best partial sentences. Better for translation; for open-ended text it produces bland, repetitive output.

Sampling: draw from p. Diverse; occasionally nonsense from the tail.

Tail control:

- **temperature** T: $p_i \propto e^{z_i/T}$. $T \to 0$ = greedy; $T > 1$ flattens. The ranking never changes. (2026: $T{=}0$ is not deterministic in practice, and reasoning models need their recommended T; for reproducibility, pin the version and run repeats.)
- **top-k**: sample only among the k most likely tokens.
- **top-p (nucleus)**: sample among the smallest set whose mass exceeds p.

After "...risks may", GPT-2:

	$T{=}0.5$	$T{=}1$	$T{=}2$
be	.68	.27	.03

The context window and the KV cache

An LLM reads at most n tokens (the **context window**): 1k (GPT-2) → 4k (GPT-3) → 128k–1M today. Everything the model "knows" at inference is its weights plus what is in the window. Nothing persists between calls.

Attention is $O(n^2)$, so long contexts are expensive; and "supports 1M tokens" does not mean "uses them well" (quality degrades in the middle of long contexts).

Generation is sequential: token t needs tokens $1..t-1$. The **KV cache** stores every previous token's keys and values so each new token costs one pass, not t.

Consequence for cost: you pay per token in, per token out; input tokens are cheap (one parallel pass), output tokens are expensive (one sequential pass each). A 10-K is ~50k tokens.

2. Pretraining at scale

What the model reads, and what size buys

The data

Pretraining corpora (Llama 3: 15 trillion tokens):

- web crawl (Common Crawl, filtered and deduplicated): the bulk
- code (GitHub): a surprisingly large share, helps reasoning
- books, Wikipedia, academic papers: small, up-weighted
- curated "high-quality" and synthetic data: growing

Filtering, deduplication and the **mixture weights** are now the main lever; the architecture barely changes.

What this means for you:

- the model has read SEC filings, FOMC statements, textbooks, papers: it "knows" finance and economics from the internet's version of them
- it has a **knowledge cut-off** and no notion of *when* a fact was true
- benchmark answers, exam questions, and **your test set** may be in there (contamination)
- rare and recent things (a 2026 ticker, a new regulation) are under-represented: this is where adaptation and retrieval come in

The objective has not changed

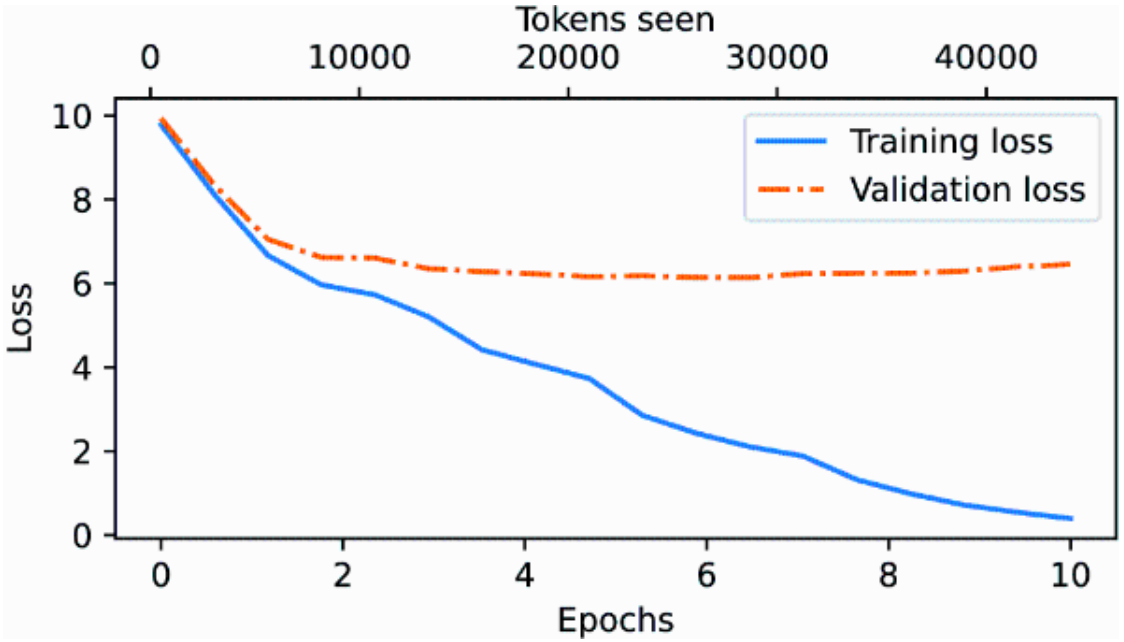
$$\mathcal{L}(\theta) = -\sum_{\text{documents}} \sum_{t} \log p_\theta(x_t \mid x_{<t})$$

Same cross-entropy as this morning, over 10^{13} tokens, on a decoder with 10^9–10^{12} parameters, for 10^{23}–10^{25} floating-point operations.

Cost of one training run:

$$C \approx 6\,N\,D \quad \text{FLOPs}$$

N parameters, D tokens (2 per parameter forward, 4 backward).



Training and validation loss fall smoothly; no phase transition in the loss. The interesting behaviour is in what the model can *do* at each loss level.

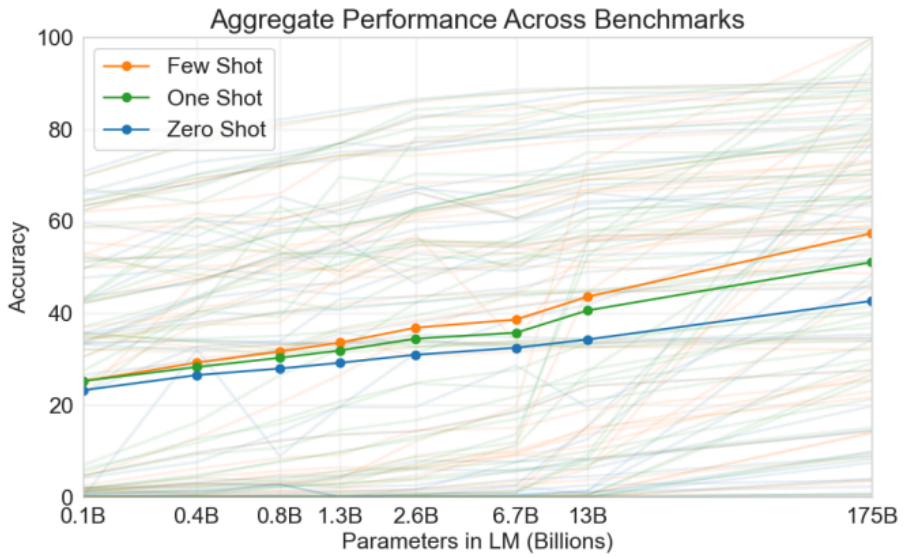
Raschka 2024, fig. 5.12 (a small model)

Model sizes, 2018–2026

model	year	params N (active)	tokens D	tokens / param
GPT-1	2018	117 M	~1 B	~8
BERT-large	2018	340 M	3.3 B	10
GPT-2	2019	1.5 B	~10 B	~7
GPT-3	2020	175 B	300 B	1.7
Chinchilla	2022	70 B	1.4 T	20
Llama 3	2024	405 B	15 T	37
DeepSeek-V3	2024	671 B (37 B)	14.8 T	22 / 400
Qwen3	2025	235 B (22 B)	36 T	153 / 1,600
Kimi K2	2025	1.0 T (32 B)	15.5 T	15 / 480
DeepSeek-V4-Pro	2026	1.6 T (49 B)	n/a	—
Kimi K3	2026	2.8 T (104 B)	n/a	—
GPT-4/5, Claude, Gemini	2023–	undisclosed	undisclosed	—

Three orders of magnitude in parameters and four in data. From 2024 the big open models are **mixtures of experts**: only a few percent of the parameters are *active* per token, so "tokens per parameter" has two readings — that is part 3.

What size buys: in-context learning



GPT-3 (Brown et al. 2020): describe a task in the prompt, optionally with a few examples, and the model does it, **with no weight update**.

- zero-shot: instruction only
- few-shot: instruction + k examples

Larger models use the examples far better. This is the behaviour that made "prompting" a thing.

One reading (Xie et al. 2022): the prompt acts as evidence about *which* of the many tasks in the pretraining mixture is being asked for; the model's output is the posterior predictive. Inference, not learning.

Brown et al. 2020, fig. 1.2, via d2l.ai

What it felt like: entity resolution, 2022

A problem every applied economist has: **are these two records the same firm?**

- *Chi Mei Optoelectronics / CMO / Chimei Innolux* (post-2010 merger)
- *Samsung Electronics / Samsung SDI* (not the same)
- *LG.Philips LCD / LG Display* (renamed 2008)

The standard pipeline, decades of work: **blocking** (cheap keys to find candidate pairs) + **matching** (string similarity, Fellegi–Sunter, a trained classifier), then hand review of the tail. Renames, mergers, subsidiaries, transliterations are exactly what slides through — and exactly what matters for the panel.

Narayan, Chami, Orr, Arora & Ré (VLDB 2022), *Can foundation models wrangle your data?*: GPT-3 with ~10 examples in the prompt, **no fine-tuning**, beat Ditto — the fine-tuned state of the art — on 4 of 7 entity-matching benchmarks. The pairs it got right were the ones the pipeline could not: the knowledge was not in the strings.

That is what the few-shot curve on the previous slide *means*: a task with its own literature became a prompt.

In 2026 a 4B open-weight model on a laptop does this. Whether it is *right* is still your job (part 6) — but the cost of the first pass fell by three orders of magnitude.

3. Scaling laws

How to spend a fixed budget on parameters versus data

Chinchilla: the allocation problem

Kaplan et al. (2020): held-out loss falls as a smooth **power law** in parameters, data and compute — smoothly enough to fit on small runs and forecast the big one. Hoffmann et al. (2022) fit the joint response surface

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

and minimize it subject to $6ND = C$: a constrained optimization with isoquants. Result: $N^{\star} \propto C^{a}$, $D^{\star} \propto C^{b}, a \approx b \approx 0.5$ — **scale parameters and data together**, about **20 tokens per parameter**.

GPT-3 had used 1.7 tokens per parameter; Chinchilla (70B, 1.4T tokens) beat Gopher (280B) at equal compute.

Two caveats worth teaching:

- it optimizes **training** cost only. Serving to millions makes a smaller model trained longer cheaper overall — hence Llama 3 at 37 tokens/param.
- Besiroglu et al. (2024) re-fitted Hoffmann's own data and got different constants; the printed fit does not reproduce the paper's headline model. A **fitted response surface, not a law.**

Lab: `tutorial/labs/scaling-law-lab.html` — move α, β and watch the optimum move.

"Emergent abilities" and the choice of metric

Wei et al. (2022): some abilities (arithmetic, multi-step reasoning) appear to be **absent below a size threshold and present above it** — a sharp jump on a smooth loss curve.

Schaeffer, Miranda & Koyejo (2023): the jumps are largely an artifact of **discontinuous metrics**. Exact-match accuracy on a 5-digit sum is 0 until every digit is right; per-token log-likelihood of the correct answer improves smoothly all along.

Why this matters to an econometrician:

- what looks like a phase transition may be a step function applied to a smooth latent
- always ask what the *score* is before believing a curve
- and conversely: a smooth loss does not mean nothing qualitative changes in what you can *use* the model for

> The same lesson as this morning's proper-scoring slide: choose the metric before the model, and report a continuous one alongside the headline accuracy.

4. From completer to assistant

Post-training: SFT, preferences, RLHF, DPO

A pretrained model completes; it does not answer

Prompt a base model with

> "What are the main climate risks for an airline?"

and a likely continuation is

> "What are the main climate risks for a bank? What are the main climate risks for ..."

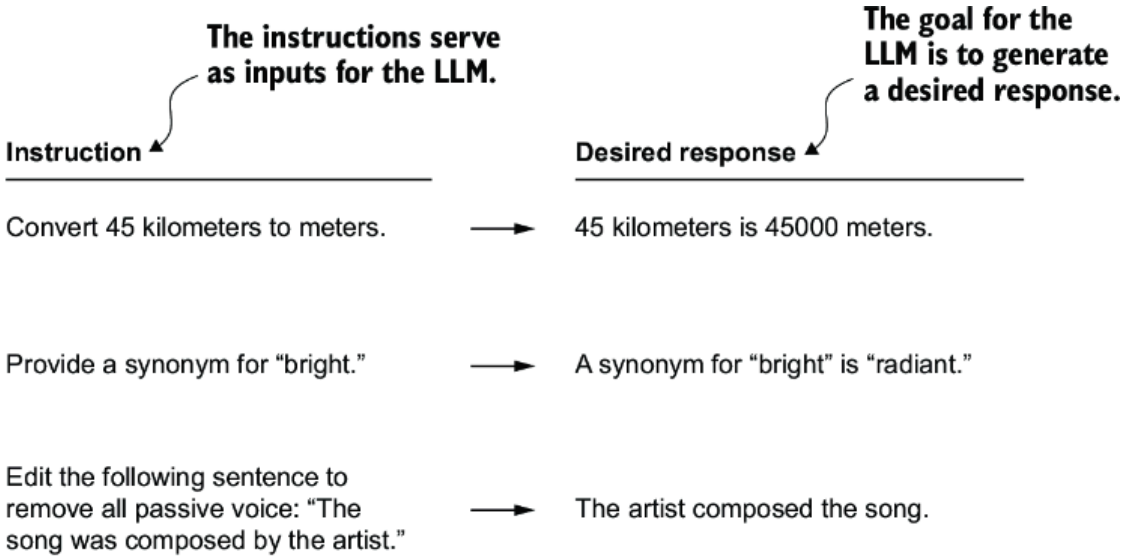
because on the internet, questions are followed by more questions. The model is doing exactly what it was trained to do.

Post-training changes the **distribution of responses**, not the model's knowledge:

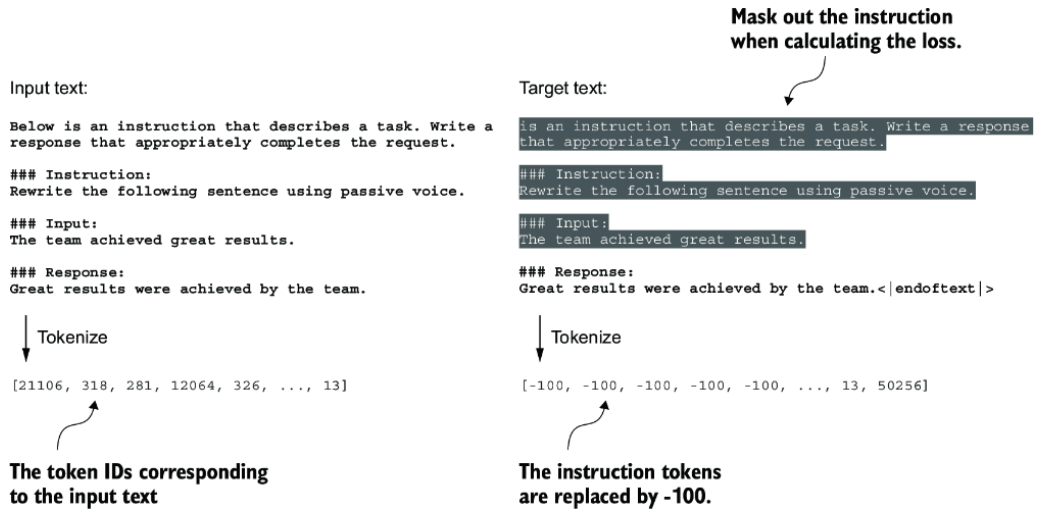
1. **Supervised fine-tuning (SFT)** on demonstrations: what a good answer looks like
2. **Preference tuning** on comparisons: which of two answers people prefer
3. (safety, tool use, reasoning traces: same machinery, different data)

The chat model you download is base + these steps. You inherit its helpfulness and its habits together.

Step 1: supervised fine-tuning on demonstrations



Collect (instruction, ideal response) pairs — thousands to hundreds of thousands, written by contractors or generated by a stronger model — and continue training with the **same next-token loss**, on the response tokens only.



Nothing new mathematically: it is the pretraining objective on a different corpus. What changes is the *conditional* the model learns: $p(\text{response} \mid \text{instruction})$.

Cost: hours on one GPU for a 7B model. Anyone can do this.

Raschka 2024, figs. 7.2, 7.13

Step 2: learning what people prefer

Show annotators two responses to the same prompt, record which they prefer. Fit a **reward model** $r_\phi(x, y)$ by the Bradley–Terry model:

$$P(y_w \succ y_l \mid x) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)$$

A logit on the reward *difference* — the same model economists use for paired comparisons and discrete choice.

Only differences at a fixed prompt are identified. Keep that in mind for the DPO slide.

Then improve the policy π_θ against the reward, **without straying too far from the SFT model** π_{ref}:

$$\max_\theta \; \mathbb{E}_{y\sim\pi_\theta}\big[r_\phi(x, y)\big] - \beta\,\mathrm{KL}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big)$$

The KL term is the leash: the reward is a *fitted proxy*, and optimizing a proxy hard enough breaks it (Goodhart). β prices the divergence.

The closed form, and DPO

The KL-regularized objective has an exact solution:

$$\pi^\star(y \mid x) = \frac{1}{Z(x)}\,\pi_{\text{ref}}(y \mid x)\, e^{\,r(x,y)/\beta}$$

$Z(x) = \sum_y \pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}$ is the **normalizer** — a sum over every possible response, so it depends on the prompt only and is never computed. The aligned model is the SFT model **re-weighted** by $e^{r/\beta}$: every response keeps positive probability; nothing is created, only tilted. $\beta \to \infty$: no change; $\beta \to 0$: put everything on the best response.

Direct Preference Optimization (Rafailov et al. 2023): invert the closed form,

$$r(x,y) = \beta \log \frac{\pi^\star(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x),$$

substitute into Bradley–Terry. $Z(x)$ **cancels** in the difference (both responses share the prompt), so you can fit the policy directly on preference pairs by logistic regression — no reward model, no RL loop.

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\!\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

Lab: `tutorial/labs/dpo-preference-lab.html` shows the cancellation numerically, and what does *not* cancel (the length exploit).

What you actually download

checkpoint	trained with	good for	watch out
base	pretraining only	completion, few-shot in the prompt, research on the raw distribution	will not follow instructions
instruct / SFT	+ demonstrations	following instructions, formats	can be confidently wrong
chat / aligned	+ preferences (RLHF/DPO)	assistants, agents, refusals	verbose; tuned to please; refusals bite on finance text
reasoning	+ RL on verifiable answers, long "thinking" traces	math, code, multi-step tasks	10–100× tokens; slower

Open weights (Llama, Qwen, Mistral, Gemma, DeepSeek) come in all four flavours and can be run and fine-tuned locally. API models come aligned only.

5. Adapting on a budget

Prompting, fine-tuning, retrieval, and when to use which

Four ways to put domain signal into a model

	what changes	cost	fixes
prompting	the conditioning set, for one call	nothing stored	task specification
retrieval (RAG)	the evidence in the context	an index + a retriever	missing or dated facts
fine-tuning (full or LoRA)	the weights	GPU hours + labels	style, register, task boundary
continued pretraining	the weights, on unlabelled domain text	GPU days	domain vocabulary and register

Most working systems are mixtures. **The failure you can measure picks the axis**: a wrong fact needs evidence, not gradient steps; a wrong format needs examples in the prompt, not a new model.

Prompting: what it is and is not

A prompt sets the conditioning:
$p(y \mid \text{instruction}, \text{examples}, x)$.

Reliable gains:

- **definitions and examples** pin down the task (few-shot)
- **format constraints** (JSON, a label set)
- **"think step by step"** (chain-of-thought) for multi-step problems: the model writes intermediate tokens it can then condition on

Limits:

- an instruction cannot create knowledge the model never saw
- **paraphrase sensitivity**: change the wording, the label distribution shifts. If the number in your paper depends on the prompt, the prompt is part of the instrument — report it, freeze it, test variants
- **prompt injection**: the crude version — "ignore previous instructions" inside a document — no longer works on frontier models (2026), but still does on small open-weight ones; the subtle versions matter once the model *acts* on documents (tools, agents)

LoRA: fine-tune 0.5 % of the weights

Full fine-tuning of a 70B model needs ~1 TB of optimizer state. Low-Rank Adaptation (Hu et al. 2022) freezes W_0 and learns a rank-r update:

$$W = W_0 + BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll d$$

$d = k = 4096$, $r = 8$: 0.4 % of the matrix trains. B starts at zero, so training begins exactly at W_0; the adapter is a few MB you can swap in and out.

What a rank sweep diagnoses: how much *capacity* the task needs. Most style/format tasks saturate at $r = 4$–16.

What rank does **not** do: bound how far the weights move. $\|BA\|$ can be arbitrarily large at $r = 1$. LoRA is a memory device, not a safety device.

With 4-bit quantization of W_0 (QLoRA) a 7B model fine-tunes on a laptop GPU; a 70B on one 48 GB card.

> Lab: `tutorial/labs/lora-rank-lab.html`.

Continued pretraining: more domain text is not automatically better

Domain-adaptive pretraining (Gururangan et al. 2020): keep training the base model's own objective on unlabelled in-domain text (filings, papers, reports). No labels needed; the aim is a better representation of the domain register.

FinBERT, BloombergGPT (50B, trained from scratch on 363B finance tokens + 345B general), ClimateBERT.

A measured counterexample (Leippold's group, Climate-ModernBERT, 2026): one encoder, three candidate corpora, nine climate benchmarks.

- academic text alone: +1.8 F1 over base
- add synthetic: +1.3
- add web text too: +0.6, and one task dropped 12 points

Out-of-register text damaged the signal. And averaging three separately-trained checkpoints beat training on the union.

Corpus choice has a **sign**; it must be validated on a locked task suite, before the adapter is built.

6. Evaluation and honest use

Benchmarks, calibration, and LLMs in empirical work

Evaluating a language model

Intrinsic: perplexity on held-out text. Cheap, continuous, comparable only within one tokenizer.

Benchmarks: a two-year half-life. MMLU (2020, exam questions) and GSM8K were saturated by 2024; the 2025–26 generation — GPQA Diamond, Humanity's Last Exam, SWE-bench Verified, FrontierMath, ARC-AGI — is on the same clock. Finance sets: FinQA, FinanceBench, FPB sentiment.

- multiple-choice accuracy hides calibration
- **contamination**: public benchmarks leak into pretraining data
- a leaderboard tells you the model is *good at that benchmark*

Judged: pairwise human preference (Arena), or a strong LLM grading a weak one — cheap, correlated with humans, biased toward length and its own style.

For your own task the Gentzkow–Kelly–Taddy rule holds: **build the evaluation before the model.**

- a **gold sample**, labelled by humans, drawn from *your* deployment distribution, locked before you tune anything
- a **metric chosen for the decision** — precision at the review threshold, not average accuracy
- a **baseline**: TF-IDF + logistic regression, or a fine-tuned BERT. If the LLM does not beat it, use the baseline.

The number that comes out is a probability over strings

Ask "is this firm exposed to climate risk? yes or no" and read $p(\text{"yes"})$. That is $p(\text{next token} = \text{yes} \mid \text{prompt})$ — a statement about **text**, from a model tuned to please, at a temperature someone chose. Not $P(\text{exposed} \mid \text{filing})$.

To use it as one you need a **map from language to events**, estimated and validated like any measurement.

Calibration. On your gold sample, bin the model's scores and plot observed frequency. Aligned models are typically over-confident.

Temperature scaling — one parameter fitted on held-out data — fixes calibration without touching the ranking.

Prior shift. 10 % exposed in training, 30 % in your sample: $p_T(y|x) \propto p_S(y|x)\, p_T(y)/p_S(y)$. One line, no retraining; the ranking stays, the threshold count moves.

> Lab: `tutorial/labs/calibration-lab.html`.

LLMs as annotators: measurement error, not magic

The most common use in economics: label 500,000 sentences you could never label by hand — sentiment, topic, hawkishness, exposure, forward-looking or not.

Treat the output as a **noisy measurement** of the latent label:

- estimate the error rate on a human-labelled subsample
- the errors are **not classical**: they correlate with text features (length, jargon, negation) and so with your regressors → bias, not just noise
- **design-based correction**: use the human labels to debias the LLM labels (Egami et al. 2023; Ludwig, Mullainathan & Rambachan 2025)

Protocol

1. lock the prompt, model version, temperature; save raw outputs
2. hand-label a random gold subsample (hundreds to a few thousand)
3. report agreement, calibration, and the corrected estimate
4. test prompt paraphrases; report the range
5. never let the model see the outcome variable

The model is an instrument. Validate it like one.

Worked example: 16,000 patents against physical ground truth

Igami, Kusaka, Scheidegger & Smirnova (2026), LCD panels 2001–2011:

- **text**: 15,964 USPTO patents of the seven major firms — title, abstract, first claim
- **the LLM reads each into a fixed JSON schema**: type, component, product-quality goal, manufacturing goal — an expert taxonomy, not free text
- **ground truth the field normally lacks**: 955 product launches, fab-level process upgrades, structural welfare estimates per innovation

What made it defensible:

- validated against **outcomes**, not other patent measures: welfare $R^2 \approx 0$ (counts, citations) → 12–14 %
- **falsification**: shuffle the texts across patents and re-classify — the association vanishes
- **open-weight replication**: the same prompt on Gemma-4-31B reproduces the measures
- the LLM adds *what the patent is for*, not portfolio scale

> The protocol from the previous slide, applied: locked schema, outcome validation, a falsification test, an open-weight replication.

Where the field is going (and what to ignore)

Real and durable

- reasoning via RL on verifiable rewards; long inference-time compute
- tools and agents: the model calls code, search, a database; loops until done
- retrieval as the standard way to supply evidence
- open-weight models within a year of the frontier; small models (1–8B) good enough for most classification

Discount heavily

- benchmark leaderboards without contamination checks
- "emergent" claims without the metric stated
- any parameter count for a closed model
- "the model understands X" without a held-out test of X

> Stable across all of it: a decoder trained on next-token prediction, tilted by preferences, conditioned by a prompt. Judge it as an estimator.

Taking stock

step	what it does	what it does not do
pretraining	learns language and a compressed internet	know today's facts; know *when* things were true
scaling	lowers loss predictably; unlocks in-context learning	change the objective
SFT	teaches the response format	add knowledge
RLHF / DPO	tilts toward preferred responses	make responses true
prompting	specifies the task	store anything
LoRA / DAPT	adapts register and task boundary cheaply	replace evidence
retrieval	supplies evidence	teach the model to read it well

Through all of it, the model outputs $p(\text{next token} \mid \text{context})$. Whenever a number comes out, ask: **which event is this a probability about?**

> 13:30 — tutorial: build these pieces yourself on Nuvolos.

References

- Brown et al. (2020). Language models are few-shot learners (GPT-3). · Kaplan et al. (2020). Scaling laws for neural language models. · Hoffmann et al. (2022). Training compute-optimal LLMs (Chinchilla). · Besiroglu et al. (2024). Chinchilla scaling: a replication attempt. · Dubey et al. (2024). The Llama 3 herd of models.
- Wei et al. (2022). Emergent abilities of LLMs. · Schaeffer, Miranda & Koyejo (2023). Are emergent abilities a mirage? · Xie et al. (2022). An explanation of in-context learning as implicit Bayesian inference.
- Ouyang et al. (2022). Training LMs to follow instructions with human feedback (InstructGPT). · Christiano et al. (2017). · Rafailov et al. (2023). Direct preference optimization. · Wei et al. (2022b). Chain-of-thought prompting.
- Hu et al. (2022). LoRA. · Dettmers et al. (2023). QLoRA. · Gururangan et al. (2020). Don't stop pretraining. · Wu et al. (2023). BloombergGPT.
- Guo et al. (2017). On calibration of modern neural networks. · Saerens, Latinne & Decaestecker (2002). Adjusting outputs to new a priori probabilities.
- Egami, Hinck, Stewart & Wei (2023). Using imperfect surrogates for downstream inference. *NeurIPS*. · Ludwig, Mullainathan & Rambachan (2025). Large language models: an applied econometric framework. · Gentzkow, Kelly & Taddy (2019). Text as data. *JEL*.

Textbooks. Amidi & Amidi, *Super Study Guide: Transformers and LLMs*, 2024, ch. 4–5. · Raschka, *Build a LLM (From Scratch)*, 2024, ch. 5–7. · Jurafsky & Martin, *SLP* 3rd ed., ch. 10–12.